



Informe de evidencia de aprendizaje

S07 Creación y Arquitectura de la SPA

SITRA-ORO

Sistema de Información, Trazabilidad y Liquidación en Acopio de Oro

1. Datos del estudiante

Nombre	Faijo Calisaya Helio Paul
Equipo	Equipo 05
Sesión	S07 - Creación y Arquitectura de la SPA
Rol o aporte	Arquitectura base SPA modular en Angular (core/shared/features), maquetación de Layout con navegación y página de inicio en /, ApiService centralizado, interceptor HTTP con X-Trace-ID (crypto.randomUUID()) y CRUD completo independiente de Mineros conectado a Spring Boot y Oracle.
GitHub	https://github.com/helionelio900-hub/SysProyec_Acopus

2. Propósito

Esta actividad demuestra la implementación individual y autónoma de la arquitectura base de una Single Page Application (SPA) modular bajo las mejores prácticas de Angular y el desarrollo de un CRUD completo sobre la tabla independiente **Minero** del dominio **SITRA-ORO** (módulo de acopio y trazabilidad de oro), conectado a un backend real en Spring Boot y base de datos Oracle Database XE.

En todas las capturas debe verse la ventana completa, el reloj con fecha y hora y el usuario o perfil visible sin recortar. Las fechas deben ser coherentes con el historial de commits del repositorio.

3. Evidencia técnica

3.1 Proyecto y arquitectura

El proyecto frontend se organizó bajo la arquitectura limpia core/shared/features:

- core/: Aloja artefactos globales singleton como ApiService (URL base), traceIdInterceptor (trazabilidad), LayoutComponent (estructura persistente) e InicioComponent (página raíz).
- shared/: Contiene modelos e interfaces transversales (models/api-error.ts).
- features/: Agrupa módulos de negocio funcionales. Para esta sesión se implementó features/acopio/mineros/ (servicio, interfaces, listado y formulario reactivo).

```
export const routes: Routes = [
  {
    path: '',
    loadComponent: () => import('./core/layout/layout').then((m) => m.Layout),
    children: [
      { path: '', title: 'Inicio | SITRA-ORO', loadComponent: () => import('./core/inicio/inicio').then((m) => m.Inicio),
        { path: 'acopio/mineros', title: 'Mineros | SITRA-ORO', loadComponent: () => import('./features/acopio/mineros').then((m) => m.Mineros),
        { path: 'acopio/mineros/nuevo', title: 'Registrar minero | SITRA-ORO', loadComponent: () => import('./features/acopio/mineros/nuevo').then((m) => m.RegistrarMinero),
        { path: 'acopio/mineros/:id/editar', title: 'Editar minero | SITRA-ORO', loadComponent: () => import('./features/acopio/mineros/editar').then((m) => m.EditarMinero),
      ],
  },
],
```

```
{ path: '**', redirectTo: '' },  
];
```

Captura 1. Estructura de carpetas core/shared/features

Abrir el proyecto en VS Code y mostrar la estructura de carpetas core/shared/features, dejando visibles el reloj con fecha y hora y el usuario.



[RECUADRO PARA INSERTAR / PEGAR CAPTURA 1]

Estructura de carpetas core/shared/features en VS Code

Abrir el explorador de VS Code mostrando las carpetas core, shared y features, el archivo app.routes.ts y la terminal compilando exitosamente.

Recordatorio: Mostrar ventana completa con reloj (fecha y hora) y usuario/perfil visible sin recortar

Explicación. La estructura modular separa los servicios transversales (core) de los módulos funcionales de negocio (features/acopio/mineros). app.routes.ts configura el Layout persistente como ruta base anidando las rutas hijas.

3.2 Layout y navegación

El componente Layout mantiene fijo el encabezado corporativo (SITRA-ORO) y el sidebar lateral de navegación, proyectando el contenido de cada pantalla dentro de `<router-outlet />`.

- **Página de inicio real en /:** Contiene un Hero informativo con descripción operativa y acceso directo, sin redirecciones automáticas.
- **Navegación reactiva:** Enlaces con `routerLinkActive="active"` para marcar visualmente la opción activa sin provocar recargas del navegador.

Captura 2. Página de inicio real en / (sin redirect)

Ejecutar la aplicación en `http://localhost:4200/` y mostrar el encabezado, sidebar y la página de inicio real, con reloj y usuario visibles.



Explicación. La pantalla en `/` presenta la vista de inicio real de la SPA sin redireccionar hacia otra URL, cumpliendo la especificación de diseño modular.

Captura 3. Navegación hacia la ruta hija /acopio/mineros

Pulsar sobre 'Mineros' en el sidebar y mostrar la transición hacia `/acopio/mineros` manteniendo el Layout intacto, con reloj y usuario visibles.



Explicación. El encabezado y sidebar se mantienen intactos durante la navegación; únicamente se reemplaza el contenido del outlet, logrando una navegación fluida e instantánea.

3.3 Servicio HTTP

La lógica de comunicación HTTP se aísla mediante servicios dedicados e interceptores funcionales:

```
// core/interceptors/trace-id.interceptor.ts
export const traceIdInterceptor: HttpInterceptorFn = (request, next) =>
  next(request.clone({ setHeaders: { 'X-Trace-ID': crypto.randomUUID() } })));

// core/api/api.service.ts
@Injectable({ providedIn: 'root' })
export class ApiService {
  readonly baseUrl = environment.apiUrl; // http://localhost:8081
  buildUrl(path: string): string { return `${this.baseUrl}${path}`; }
}

// features/acopio/mineros/minero.service.ts
@Injectable({ providedIn: 'root' })
export class MineroService {
  private readonly http = inject(HttpClient);
  private readonly url = inject(ApiService).buildUrl('/api/v1/acopio/mineros');
  listar(): Observable<Minero[]> { return this.http.get<Minero[]>(this.url); }
  crear(request: MineroRequest): Observable<Minero> { return this.http.post<Minero>(this.url, request); }
  actualizar(id: number, request: MineroRequest): Observable<Minero> { return this.http.put<Minero>(`${this.url}/${id}`, request); }
  eliminar(id: number): Observable<void> { return this.http.delete<void>(`${this.url}/${id}`); }
}
```

Captura 4. Código fuente de traceIdInterceptor y MineroService

Abrir trace-id.interceptor.ts y minero.service.ts en VS Code y mostrar el interceptor y los métodos del CRUD, con reloj y usuario visibles.



[RECUADRO PARA INSERTAR / PEGAR CAPTURA 4]

Código fuente de traceIdInterceptor, ApiService y MineroService

Mostrar en VS Code el interceptor inyectando X-Trace-ID, ApiService con baseUrl y MineroService con llamadas tipadas.

Recordatorio: Mostrar ventana completa con reloj (fecha y hora) y usuario/perfil visible sin recortar

Explicación. traceIdInterceptor inyecta un UUID en X-Trace-ID en cada petición. Los componentes no llaman a HttpClient directamente, sino a través de MineroService.

Captura 5. Pestaña Network: Header X-Trace-ID y respuesta 200 OK

Abrir DevTools Network en el navegador, inspeccionar la petición a `/api/v1/acopio/mineros` y mostrar X-Trace-ID en Request Headers y status 200 OK.

**[RECUADRO PARA INSERTAR / PEGAR CAPTURA 5]****Inspección Network: Request Header X-Trace-ID y status 200 OK**

Mostrar en Chrome DevTools la petición a `http://localhost:8081/api/v1/acopio/mineros` con cabecera X-Trace-ID y respuesta 200 OK del backend.

Recordatorio: Mostrar ventana completa con reloj (fecha y hora) y usuario/perfil visible sin recortar

Explicación. Se comprueba que el interceptor adjunta la cabecera de trazabilidad X-Trace-ID y que la petición al backend real en Spring Boot responde con código 200 OK.

3.4 CRUD independiente

Mínero es una **tabla independiente** de SITRA-ORO: contiene los datos del productor minero y no requiere seleccionar previamente ninguna entidad foránea para crearse.

Captura 6. CRUD Caso 1 - Listar mineros desde backend real

Abrir `/acopio/mineros` y mostrar la tabla cargando los mineros reales desde la base de datos Oracle, con reloj y usuario visibles.



[RECUADRO PARA INSERTAR / PEGAR CAPTURA 6]

CRUD Caso 1 - Listar mineros desde backend real

Mostrar la tabla con mineros registrados en Oracle (FaijoCalisayaHelioPaul, EloyFaijoQuispe, etc.), documento, zona y acciones.

Recordatorio: Mostrar ventana completa con reloj (fecha y hora) y usuario/perfil visible sin recortar

Explicación. La tabla se llena a partir del endpoint GET `/api/v1/acopio/mineros` conectado a Oracle XE, administrado reactivamente con Signals.

Captura 7. CRUD Caso 2 - Crear nuevo minero (POST)

Llenar el formulario en `/acopio/mineros/nuevo` con datos válidos y guardar, dejando visibles los campos, el reloj y el usuario.



[RECUADRO PARA INSERTAR / PEGAR CAPTURA 7]

CRUD Caso 2 - Crear nuevo minero (POST)

Mostrar el formulario de registro con documento (mínimo 8 dígitos), nombres, teléfono y zona antes de enviar la petición POST.

Recordatorio: Mostrar ventana completa con reloj (fecha y hora) y usuario/perfil visible sin recortar

Explicación. Formulario reactivo que valida campos obligatorios y envía la petición POST hacia el backend, redirigiendo al listado tras una persistencia exitosa.

Captura 8. CRUD Caso 3 - Editar minero existente (GET by ID + PUT)

Abrir `/acopio/mineros/:id/editar`, mostrar los datos precargados desde el backend, modificar un valor y guardar, con reloj y usuario visibles.

**[RECUADRO PARA INSERTAR / PEGAR CAPTURA 8]****CRUD Caso 3 - Editar minero existente (GET by ID + PUT)**

Mostrar el formulario precargado con datos del minero mediante GET by ID y la modificación del teléfono o zona de procedencia.

Recordatorio: Mostrar ventana completa con reloj (fecha y hora) y usuario/perfil visible sin recortar

Explicación. El formulario recupera la entidad por su identificador con GET `/api/v1/acopio/mineros/{id}`, aplica `patchValue` y persiste las modificaciones con PUT.

Captura 9. CRUD Caso 4 - Eliminar minero (DELETE con reactividad)

Eliminar un minero de prueba mediante el botón Eliminar y mostrar la actualización reactiva de la tabla sin recargar la página, con reloj y usuario visibles.

**[RECUADRO PARA INSERTAR / PEGAR CAPTURA 9]****CRUD Caso 4 - Eliminar minero (DELETE con reactividad)**

Mostrar la ejecución del DELETE hacia el backend y la confirmación inmediata en la tabla removiendo la fila del estado local.

Recordatorio: Mostrar ventana completa con reloj (fecha y hora) y usuario/perfil visible sin recortar

Explicación. La acción ejecuta DELETE `/api/v1/acopio/mineros/{id}`. Al confirmarse el código 204 No Content, se actualiza la señal reactiva sin recargar la página.

4. Error o hallazgo técnico**Validación de longitud de documento y prevención en cliente**

Durante las pruebas de integración con el backend, al ingresar documentos de prueba con pocos dígitos (ej. 123), el backend en Spring Boot rechazaba la solicitud respondiendo con error 400 Bad Request debido a la anotación `@Size(min = 8, max = 15)` de Jakarta Validation. En el frontend original no existía validador de longitud mínima local, enviando solicitudes destinadas al rechazo.

Solución implementada: Se incorporó `Validators.minLength(8)` y `Validators.maxLength(15)` en el `FormBuilder`, junto con una función auxiliar `mensaje(control)` que advierte al usuario en el evento `blur` ("Debe tener al menos 8 caracteres") y bloquea el botón de envío, impidiendo emitir tráfico innecesario a la red.

Captura 10. Validación reactiva en tiempo real y prevención de envíos erróneos

Ingresar un documento con menos de 8 caracteres y salir del campo para mostrar el mensaje de validación reactivo en rojo, con reloj y usuario visibles.

**[RECUADRO PARA INSERTAR / PEGAR CAPTURA 10]****Validación reactiva en tiempo real y prevención de envíos erróneos**

Mostrar el formulario detectando documento con menos de 8 caracteres y campos obligatorios tocados, desplegando alerta visual en rojo.

Recordatorio: Mostrar ventana completa con reloj (fecha y hora) y usuario/perfil visible sin recortar

Explicación. El formulario valida en el cliente antes de llamar a la red, garantizando coherencia con las restricciones del backend y ofreciendo retroalimentación inmediata.

5. Reflexión técnica breve

¿Por qué separar `CategoriaService` (o `MineroService`) del componente que lo usa facilita un cambio futuro en la URL o en la forma de consumir el backend?

Separar `MineroService` del componente respeta el Principio de Responsabilidad Única (SRP): el componente se limita a gobernar el estado de la vista y la interacción del usuario, mientras que el servicio encapsula los detalles del protocolo de red y el contrato HTTP. Si la API cambia su versionamiento (por ejemplo, migrando de `/api/v1/acopio/mineros` a `/api/v2/mineros`), cambia de dominio base, incorpora tokens Bearer JWT o reemplaza peticiones REST por WebSocket/GraphQL, la actualización se realiza en un único punto dentro del servicio. Ningún componente consumidor (como `MineroList` o `MineroForm`) requiere ser modificado ni recompilado, garantizando alta cohesión, bajo acoplamiento y facilidad para realizar pruebas unitarias aisladas mediante mocks.

6. Respuestas a las Preguntas de Defensa

1. **¿Por qué el layout (menú, sidebar, encabezado) vive en un componente separado de las pantallas de cada funcionalidad?**

Porque conforma la estructura común y persistente de la SPA. Al aislarlo en `LayoutComponent`, las vistas hijas se proyectan dinámicamente en el `<router-outlet />`, evitando duplicar código de maquetación en cada pantalla, previniendo reinicializaciones del DOM global y manteniendo el estado de navegación.

2. **¿Qué diferencia hay entre organizar componentes por tipo y organizarlos por funcionalidad (core/shared/features)?**

Organizar por tipo agrupa todos los componentes juntos y todos los servicios juntos, volviendo el proyecto inmanejable al crecer. Organizar por funcionalidad (core/shared/features) sigue los principios de Domain-Driven Design (DDD): cada módulo es autónomo, cohesivo, fácil de auditar, propicio para carga perezosa (lazy loading) y asignable a distintos miembros del equipo sin generar conflictos.

3. **¿Por qué `MineroService` no expone directamente el `HttpClient` a los componentes que lo usan?**

Porque rompería el principio de encapsulamiento. Si un componente llamara a `HttpClient.get()` directamente, quedaría acoplado a URLs hardcodeadas, verbos HTTP y detalles de cabeceras. El servicio actúa como una fachada que entrega datos con tipado estricto (`Observable<Minero[]>`).

4. **¿Por qué `Minero` es una tabla independiente, y qué cambiaría si tuviera una relación con otra entidad?**

Es independiente porque no requiere la existencia previa de otra entidad ni una clave foránea para crearse. Si fuera dependiente (como una liquidación o entrega de mineral), el formulario del frontend estaría obligado a cargar previamente un selector para elegir la entidad padre antes de enviar la creación.

5. **¿Por qué `ApiService` no sabe nada sobre `Minero`, y qué otro servicio futuro reutilizaría exactamente el mismo `ApiService`?**

`ApiService` reside en core/ y su responsabilidad exclusiva es proveer la URL base y utilidades de transporte global. No debe tener conocimiento del dominio de negocio. Servicios futuros como `AcopiadorService` o `LiquidacionService` reutilizarán el mismo `ApiService.buildUrl(...)` sin alterar core/.

6. **¿Por qué el interceptor HTTP agrega su header en un solo lugar, en vez de que cada servicio lo agregue? ¿Qué otro problema resuelve?**

Centraliza un aspecto transversal (Cross-Cutting Concern), evitando duplicar código propenso a errores en cada servicio. Con el mismo mecanismo se resuelven de forma elegante la inyección de tokens de autenticación JWT (`Authorization: Bearer`), la captura global de errores (401/403/500) y el manejo de spinners de carga globales.

7. **Si tu CRUD autónomo usa una tabla distinta a `Categoria`, ¿qué validaciones del backend tuviste que respetar en el frontend?**

Para `Minero` se respetaron las reglas de `MineroRequest.java`: documento de identidad obligatorio con longitud entre 8 y 15 caracteres; nombres y apellidos obligatorios con máximo 150 caracteres; teléfono con límite de 20 caracteres; y zona de procedencia con un máximo de 100 caracteres.

7. Rúbrica de evaluación

Criterio	Peso	Nivel obtenido	Puntos	Justificación técnica
1. Proyecto y arquitectura	25%	A (20 pts)	20	Estructura modular core/shared/features correcta y coherente con el dominio SITRA-ORO. Standalone components y router jerárquico.
2. Layout y navegación	25%	A (20 pts)	20	Layout funcional con header corporativo, sidebar con links activos y página de inicio real en / sin redirect (Capturas 2 y 3).
3. Servicio HTTP	25%	A (20 pts)	20	ApiService en core/, MineroService dedicado e interceptor inyectando X-Trace-ID probado contra backend real con 200 OK (Captura 5).
4. CRUD independiente	25%	A (20 pts)	20	Los 4 casos (Listar, Crear, Editar, Eliminar) funcionando de extremo a extremo contra backend real Spring Boot y Oracle XE (Capturas 6 al 9).

Nota final = $(0.25 \times 20) + (0.25 \times 20) + (0.25 \times 20) + (0.25 \times 20) = 20 / 20$



Anexo: Feedback de la sesión

Sesión S07: Creación y Arquitectura de la SPA

1. ¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?

Aprender a estructurar un frontend empresarial moderno con Angular Standalone separando responsabilidades en core, shared y features, y comprender el rol de los interceptores HTTP funcionales para inyectar trazabilidad distribuida (X-Trace-ID) sin alterar los componentes.

2. ¿Qué punto de la clase te resultó más confuso o te dejó con dudas?

Al inicio, la configuración del enrutamiento anidado para asegurar que los componentes hijos utilicen el LayoutComponent de forma persistente sin provocar parpadeos ni recargas al cambiar de pantalla.

3. ¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?

¿Cómo se gestiona adecuadamente la renovación de tokens JWT vencidos dentro de un interceptor HTTP en Angular cuando múltiples peticiones concurrentes reciben un código de error 401?

4. Sobre tu nivel de comprensión de la clase de hoy, marca una opción:

☒ ¡Entendido! - Lo domino y podría explicarlo.

☐ Más o menos. - Entendí la idea general, pero tengo dudas.

☐ Necesito ayuda. - Me siento perdido/a con este tema.

5. ¿Cómo puedo ayudarte a comprender mejor el tema?

Con ejemplos prácticos de comunicación entre componentes mediante Signals reactivas y manejo de estados de carga globales.

6. Pensando en tu participación y esfuerzo en la clase de hoy, ¿cómo te autoevaluarías?

☒ Muy Comprometido/a: Me esforcé al máximo.

☐ Comprometido/a: Sé que podría haberme esforzado un poco más.

☐ Poco Comprometido/a: Hoy no di mi mejor esfuerzo.

7. Mi satisfacción con la clase fue... (califica del 1 al 10):

10 / 10 — La sesión brindó los fundamentos esenciales para conectar la interfaz de usuario con la arquitectura backend construida en las sesiones previas.